



VINUNIVERSITY

Lab 06 TUTORIAL

Week 06, Apr 02 2026

College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring

1. Selection Sort

Selection Sort repeatedly selects the **minimum element** from the unsorted portion and moves it to the sorted portion. It is an in-place, comparison-based sorting algorithm with a time complexity of $O(n^2)$.

Example

Array: [64, 25, 12, 22]

1. Find the minimum element in the entire array: 12 → Swap with the first element → [12, 25, 64, 22]
2. Find the minimum element in the unsorted portion: 22 → Swap with the second element → [12, 22, 64, 25]
3. Find the minimum element in the unsorted portion: 25 → Swap with the third element → [12, 22, 25, 64]

The array is now sorted.

Java Implementation

```
1 public class SortingAlgorithms {
2
3     // Selection Sort
4     public static void selectionSort(int[] arr) {
5         int n = arr.length;
6
7         // Traverse through all elements
8         for (int i = 0; i < n - 1; i++) {
9             int minIndex = i;
10
11             // Find the minimum element in unsorted part
12             for (int j = i + 1; j < n; j++) {
13                 if (arr[j] < arr[minIndex]) {
14                     minIndex = j;
```

```
15         }
16     }
17
18     // Swap the found minimum element with the first element
19     swap(arr, i, minIndex);
20 }
21 }
22
23 // Helper method to swap elements
24 private static void swap(int[] arr, int i, int j) {
25     int temp = arr[i];
26     arr[i] = arr[j];
27     arr[j] = temp;
28 }
29 }
```

2. Bubble Sort

Bubble Sort repeatedly swaps adjacent elements if they are in the wrong order. It is simple but inefficient with a time complexity of $O(n^2)$.

Example

Array: [5, 1, 4, 2]

1. Compare 5 and 1 → Swap → [1, 5, 4, 2]
2. Compare 5 and 4 → Swap → [1, 4, 5, 2]
3. Compare 5 and 2 → Swap → [1, 4, 2, 5]
4. Continue iterating until the array is sorted → [1, 2, 4, 5]

Java Implementation

```
1 public class SortingAlgorithms {
2
3     // Bubble Sort
4     public static void bubbleSort(int[] arr) {
5         int n = arr.length;
6
7         // Traverse through all elements
8         for (int i = 0; i < n - 1; i++) {
9             // Last i elements are already sorted
10            for (int j = 0; j < n - i - 1; j++) {
11                if (arr[j] > arr[j + 1]) {
12                    swap(arr, j, j + 1);
13                }
14            }
15        }
16    }
17 }
```

3. Insertion Sort

Insertion Sort builds a sorted array one element at a time by repeatedly picking the next element and inserting it into its correct position within the sorted portion. It has a time complexity of $O(n^2)$.

Example

Array: [4, 3, 2]

1. Start with 4 → Sorted: [4]
2. Insert 3 → Sorted: [3, 4]
3. Insert 2 → Sorted: [2, 3, 4]

Java Implementation

```
1 public class SortingAlgorithms {
2
3     // Insertion Sort
4     public static void insertionSort(int[] arr) {
5         int n = arr.length;
6
7         // Start from second element
8         for (int i = 1; i < n; i++) {
9             int key = arr[i];
10            int j = i - 1;
11
12            // Shift elements of arr[0..i-1] that are greater than key
13            while (j >= 0 && arr[j] > key) {
14                arr[j + 1] = arr[j];
15                j--;
16            }
17
18            // Insert the key at its correct position
19            arr[j + 1] = key;
20        }
21    }
22 }
```

4. Merge Sort

Merge Sort is a **divide-and-conquer** algorithm. It recursively divides the array into two halves, sorts them, and then merges the sorted halves. It has a time complexity of $O(n \log n)$.

Example

Array: [5, 2, 4, 6]

- Split: [5, 2] and [4, 6]
- Sort Left: [2, 5]
- Sort Right: [4, 6]
- Merge: [2, 4, 5, 6]

Java Implementation

```
1 public class SortingAlgorithms {
2
3     // Merge Sort
4     public static void mergeSort(int[] arr, int left, int right) {
5         if (left >= right) {
6             return;
7         }
8
9         int mid = (left + right) / 2;
10
11         // Recursively sort left and right halves
12         mergeSort(arr, left, mid);
13         mergeSort(arr, mid + 1, right);
14
15         // Merge the sorted halves
16         merge(arr, left, mid, right);
17     }
18
19     // Merge the two halves
20     private static void merge(int[] arr, int left, int mid, int right) {
21         int[] temp = new int[right - left + 1];
22         int i = left, j = mid + 1, k = 0;
23
24         // Merge the two halves in sorted order
25         while (i <= mid && j <= right) {
26             if (arr[i] <= arr[j]) {
27                 temp[k++] = arr[i++];
28             } else {
29                 temp[k++] = arr[j++];
30             }
31         }
32
33         // Copy remaining elements of the left half
```

```
34     while (i <= mid) {
35         temp[k++] = arr[i++];
36     }
37
38     // Copy remaining elements of the right half
39     while (j <= right) {
40         temp[k++] = arr[j++];
41     }
42
43     // Copy the merged elements back into the original array
44     System.arraycopy(temp, 0, arr, left, temp.length);
45 }
46 }
```

5. QuickSort

QuickSort is a **divide-and-conquer** algorithm that works by selecting a **pivot** element, partitioning the array into elements greater than and less than the pivot, and then recursively sorting the sub-arrays. It has an average time complexity of $O(n \log n)$.

Example

Array: [7, 10, 4, 3, 20, 15]

1. Choose 15 as the pivot. 2. Partition the array around the pivot: [7, 10, 4, 3] | 15 | [20] 3. Recursively sort the left sub-array [7, 10, 4, 3]:

Choose pivot: 4 $\Rightarrow$ [3, 4, 7, 10]

4. The sorted array is [3, 4, 7, 10, 15, 20].

Java Implementation

```
1 public class SortingAlgorithms {
2
3     // QuickSort Algorithm
4     public static void quickSort(int[] arr, int low, int high) {
5         if (low < high) {
6             // Partition the array
7             int pi = partition(arr, low, high);
8
9             // Recursively sort the sub-arrays
10            quickSort(arr, low, pi - 1);
11            quickSort(arr, pi + 1, high);
12        }
13    }
14
15    // Partition the array around the pivot
16    private static int partition(int[] arr, int low, int high) {
17        int pivot = arr[high];
18        int i = low - 1;
19
20        for (int j = low; j < high; j++) {
21            if (arr[j] <= pivot) {
22                i++;
23                swap(arr, i, j);
24            }
25        }
26
27        swap(arr, i + 1, high);
28        return i + 1;
29    }
30
31    // Helper method to swap elements
32    private static void swap(int[] arr, int i, int j) {
33        int temp = arr[i];
34        arr[i] = arr[j];
```

```
35         arr[j] = temp;  
36     }  
37 }
```